

ASSIGNMENT-12.5

H.no-2303A51641

Batch – 29

Task 1 – Sorting – Merge Sort Implementation

Prompt :

#generate a Python program that implements the Merge Sort algorithm.create a function merge_sort(arr) that sorts a list in ascending order.include time complexity and space complexity in the function docstring.

CODE:

```
1 #Generate a Python program that implements the Merge Sort algorithm.create a function merge_sort(arr) that sorts a list in ascending order.include time complexity and space complexity in the function.
2 def merge_sort(arr):
3     """
4     Sorts a list in ascending order using the Merge Sort algorithm.
5
6     Time Complexity: O(n log n) - where n is the number of elements in the list.
7     Space Complexity: O(n) - due to the temporary arrays used for merging.
8
9     Parameters:
10    arr (list): The list to be sorted.
11
12    Returns:
13    list: A new sorted list in ascending order.
14    """
15    if len(arr) <= 1:
16        return arr
17
18    mid = len(arr) // 2
19    left_half = merge_sort(arr[:mid])
20    right_half = merge_sort(arr[mid:])
21
22    return merge(left_half, right_half)
23
24 def merge(left, right):
25     """Merges two sorted lists into a single sorted list.
26
27     Parameters:
28     left (list): The first sorted list.
29     right (list): The second sorted list.
30
31     Returns:
32     list: A merged sorted list containing all elements from both input lists.
33     """
34     merged = []
35     left_index = right_index = 0
36
37     while left_index < len(left) and right_index < len(right):
38         if left[left_index] < right[right_index]:
39             merged.append(left[left_index])
40             left_index += 1
41         else:
42             merged.append(right[right_index])
43             right_index += 1
44
45     # If there are remaining elements in the left half, add them to merged
46     while left_index < len(left):
47         merged.append(left[left_index])
48         left_index += 1
49
50     # If there are remaining elements in the right half, add them to merged
51     while right_index < len(right):
52         merged.append(right[right_index])
53         right_index += 1
54
55     return merged
56
57 # Example usage:
58 if __name__ == "__main__":
59     unsorted_list = [38, 27, 43, 3, 9, 82, 10]
60     sorted_list = merge_sort(unsorted_list)
61     print("Sorted list:", sorted_list)
```

OUTPUT:

```
Sorted list: [3, 9, 10, 27, 38, 43, 82]
PS D:\AI>
```

JUSTIFICATION:

Merge sort is used because it divides the list into smaller parts and then merges them after sorting. It works efficiently for large data. The time complexity is $O(n \log n)$ which makes it faster than simple sorting methods like bubble sort. **Task 2 – Searching – Binary Search with AI Optimization**

Prompt :

#create a binary search function that finds a target element in a sorted list.create a function `binary_search(arr,target)` returning the index of the target or -1 if not found.

#Include docstrings explaining best, average, and worst-case complexities.

CODE:

```
58 #create a binary search function that finds a target element in a sorted list.create a function binary_search(arr,target) returning the index of the target or -1 if not found.
59 #Include docstrings explaining best, average, and worst-case complexities.
60 def binary_search(arr, target):
61     """
62     Performs binary search to find the index of the target element in a sorted list.
63
64     Best Case Complexity:  $O(1)$  - when the target is at the middle of the list.
65     Average Case Complexity:  $O(\log n)$  - where n is the number of elements in the list.
66     Worst Case Complexity:  $O(\log n)$  - when the target is not found or is at the end of the list.
67
68     Parameters:
69     arr (list): The sorted list to search through.
70     target: The element to find in the list.
71
72     Returns:
73     int: The index of the target element if found, otherwise -1.
74     """
75     left, right = 0, len(arr) - 1
76
77     while left <= right:
78         mid = left + (right - left) // 2
79
80         # Check if the target is present at mid
81         if arr[mid] == target:
82             return mid
83         # If target is greater, ignore the left half
84         elif arr[mid] < target:
85             left = mid + 1
86         # If target is smaller, ignore the right half
87         else:
88             right = mid - 1
89
90     # Target was not found in the list
91     return -1
92
93 # Example usage:
94 if __name__ == "__main__":
95     sorted_list = [3, 9, 10, 27, 38, 43, 82]
96     target = 10
97     index = binary_search(sorted_list, target)
98     if index != -1:
99         print(f"Target {target} found at index: {index}")
100     else:
101         print(f"Target {target} not found in the list.")
```

OUTPUT:

```
Target 10 found at index: 2
PS D:\AI>
```

JUSTIFICATION:

Binary search is used to find an element in a sorted list. It works by dividing the list into halves and checking the middle element. This makes searching faster with $O(\log n)$ time complexity.

Task 3 – Smart Healthcare Appointment Scheduling System

Prompt :

#A healthcare platform maintains appointment records containing #
appointment ID, patient name, doctor name, appointment time, and
consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

recommend suitable searching and sorting algorithms.

Justify the selected algorithms.

Implement the algorithms in Python.

CODE:

```
186 #A healthcare platform maintains appointment records containing
187 # appointment ID, patient name, doctor name, appointment time, and
188 # consultation fee. The system needs to:
189 # 1. Search appointments using appointment ID.
190 # 2. Sort appointments based on time or consultation fee.
191 # recommend suitable searching and sorting
192 # algorithms.
193 # Justify the selected algorithms.
194 # Implement the algorithms in Python.
195
196 '''For searching appointments using appointment ID, a suitable algorithm would be Binary Search, provided that the appointment records are sorted by appointment ID. Binary Search is efficient with
197 For sorting appointments based on time or consultation fee, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of  $O(n \log n)$  and is stable, meaning it preserves the rela
198 Here is the implementation of both algorithms in Python:'''
199
200 class Appointment:
201     def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
202         self.appointment_id = appointment_id
203         self.patient_name = patient_name
204         self.doctor_name = doctor_name
205         self.appointment_time = appointment_time
206         self.consultation_fee = consultation_fee
207
208 def binary_search_appointments(appointments, target_id):
209     """
210     Searches for an appointment by appointment ID using Binary Search.
211
212     Time Complexity:  $O(\log n)$  - where  $n$  is the number of appointments.
213     Space Complexity:  $O(1)$  - as it uses constant extra space.
214
215     Parameters:
216     appointments (list): A list of Appointment objects sorted by appointment_id.
217     target_id: The appointment ID to search for.
218
219     Returns:
220     Appointment: The appointment object if found, otherwise None.
221     """
222     left, right = 0, len(appointments) - 1
223
224     while left <= right:
225         mid = left + (right - left) // 2
226
227         if appointments[mid].appointment_id == target_id:
228             return appointments[mid]
229         elif appointments[mid].appointment_id < target_id:
230             left = mid + 1
231         else:
232             right = mid - 1
233
234     return None
235
236 def merge_sort_appointments(appointments, key):
```

```

154 Sorts a list of Appointment objects based on a specified key using Merge Sort.
155 Time Complexity:  $O(n \log n)$  - where  $n$  is the number of appointments.
156 Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
157 Parameters:
158 appointments (list): A list of Appointment objects to be sorted.
159 key (str): The attribute to sort by ('appointment_time' or 'consultation_fee').
160 Returns:
161 list: A new sorted list of Appointment objects based on the specified key.
162 """
163 if len(appointments) <= 1:
164     return appointments
165
166 mid = len(appointments) // 2
167 left_half = merge_sort_appointments(appointments[:mid], key)
168 right_half = merge_sort_appointments(appointments[mid:], key)
169
170 return merge_appointments(left_half, right_half, key)
171 def merge_appointments(left, right, key):
172     """Merges two sorted lists of Appointment objects based on a specified key.
173     Parameters: left (list): The first sorted list of Appointment objects.
174     right (list): The second sorted list of Appointment objects.
175     key (str): The attribute to sort by ('appointment_time' or 'consultation_fee').
176     Returns: list: A merged sorted list of Appointment objects based on the specified key.
177     """
178     merged = []
179     left_index = right_index = 0
180
181     while left_index < len(left) and right_index < len(right):
182         if getattr(left[left_index], key) < getattr(right[right_index], key):
183             merged.append(left[left_index])
184             left_index += 1
185         else:
186             merged.append(right[right_index])
187             right_index += 1
188
189     while left_index < len(left):
190         merged.append(left[left_index])
191         left_index += 1
192
193     while right_index < len(right):
194         merged.append(right[right_index])
195         right_index += 1
196
197     return merged

```

```

196     return merged
197
198 # Example usage:
199 if __name__ == "__main__":
200     appointments = [
201         Appointment(1, "Alice", "Dr. Smith", "2024-07-01 10:00", 100),
202         Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:00", 150),
203         Appointment(3, "Charlie", "Dr. Brown", "2024-07-01 09:00", 120),
204     ]
205
206     # Sort appointments by time
207     sorted_by_time = merge_sort_appointments(appointments, key='appointment_time')
208     print("Appointments sorted by time:")
209     for appt in sorted_by_time:
210         print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")
211
212     # Search for an appointment by ID
213     target_id = 2
214     found_appointment = binary_search_appointments(sorted_by_time, target_id)
215     if found_appointment:
216         print(f"Appointment found: {found_appointment.patient_name} with {found_appointment.doctor_name} at {found_appointment.appointment_time} for ${found_appointment.consultation_fee}")
217     else:
218         print(f"Appointment with ID {target_id} not found.")
219
220

```

OUTPUT:

```

Appointments sorted by time:
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Bob with Dr. Jones at 2024-07-01 11:00 for $150

```

```

Appointment found: Bob with Dr. Jones at 2024-07-01 11:00 for $150

```

```

PS D:\AI>

```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to find the appointment using appointment ID because IDs are unique and can be sorted. It helps in finding the appointment quickly.

Sorting Algorithm: Merge Sort

Merge sort is used to sort appointments based on time or consultation fee. It works efficiently for large records and gives consistent performance.

Task 4 – Railway Ticket Reservation System Scenario

Prompt :

#A railway reservation system stores booking details such as ticket #

ID, passenger name, train number, seat number, and travel date. The

system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python **CODE:**

```
222 #A railway reservation system stores booking details such as ticket
223 # ID, passenger name, train number, seat number, and travel date. The
224 # system must:
225 # 1. Search tickets using ticket ID.
226 # 2. Sort bookings based on travel date or seat number.
227 # Identify efficient algorithms using AI assistance.
228 # • Justify the algorithm choices.
229 # • Implement searching and sorting in Python.
230 '''For searching tickets using ticket ID, a suitable algorithm would be Binary Search, provided that the booking records are sorted by ticket ID. Binary Search is efficient with a time complexity of O(log n) and is stable, meaning it preserves the relative order of elements. For sorting bookings based on travel date or seat number, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of O(n log n) and is stable, meaning it preserves the relative order of elements. Here is the implementation of both algorithms in Python:'''
231
232 class Booking:
233     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
234         self.ticket_id = ticket_id
235         self.passenger_name = passenger_name
236         self.train_number = train_number
237         self.seat_number = seat_number
238         self.travel_date = travel_date
239
240 def binary_search_bookings(bookings, target_id):
241     """
242     Searches for a booking by ticket ID using Binary Search.
243
244     Time Complexity: O(log n) - where n is the number of bookings.
245     Space Complexity: O(1) - as it uses constant extra space.
246
247     Parameters:
248     bookings (list): A list of Booking objects sorted by ticket_id.
249     target_id: The ticket ID to search for.
250
251     Returns:
252     Booking: The booking object if found, otherwise None.
253     """
254     left, right = 0, len(bookings) - 1
255
256     while left <= right:
257         mid = left + (right - left) // 2
258
259         if bookings[mid].ticket_id == target_id:
260             return bookings[mid]
261         elif bookings[mid].ticket_id < target_id:
262             left = mid + 1
263         else:
264             right = mid - 1
265
266     return None
267
268 def merge_sort_bookings(bookings, key):
```



```

267 def merge_sort_bookings(bookings, key):
268     """Sorts a list of Booking objects based on a specified key using Merge Sort.
269     Time Complexity: O(n log n) - where n is the number of bookings.
270     Space Complexity: O(n) - due to the temporary arrays used for merging.
271     Parameters:
272     bookings (list): A list of Booking objects to be sorted.
273     key (str): The attribute to sort by ('travel_date' or 'seat_number').
274     Returns:
275     list: A new sorted list of Booking objects based on the specified key.
276     """
277     if len(bookings) <= 1:
278         return bookings
279
280     mid = len(bookings) // 2
281     left_half = merge_sort_bookings(bookings[:mid], key)
282     right_half = merge_sort_bookings(bookings[mid:], key)
283
284     return merge_bookings(left_half, right_half, key)
285
286 def merge_bookings(left, right, key):
287     """Merges two sorted lists of Booking objects based on a specified key.
288     Parameters:
289     left (list): The first sorted list of Booking objects.
290     right (list): The second sorted list of Booking objects.
291     key (str): The attribute to sort by ('travel_date' or 'seat_number').
292     Returns:
293     list: A merged sorted list of Booking objects based on the specified key.
294     """
295     merged = []
296     left_index = right_index = 0
297
298     while left_index < len(left) and right_index < len(right):
299         if getattr(left[left_index], key) < getattr(right[right_index], key):
300             merged.append(left[left_index])
301             left_index += 1
302         else:
303             merged.append(right[right_index])
304             right_index += 1
305
306     while left_index < len(left):
307         merged.append(left[left_index])
308         left_index += 1
309
310     while right_index < len(right):
311         merged.append(right[right_index])
312         right_index += 1
313
314     return merged

```

```

305     while left_index < len(left):
306         merged.append(left[left_index])
307         left_index += 1
308
309     while right_index < len(right):
310         merged.append(right[right_index])
311         right_index += 1
312
313     return merged
314
315 # Example usage:
316 if __name__ == "__main__":
317     bookings = [
318         Booking(1, "Alice", "Train A", "12A", "2024-08-01"),
319         Booking(2, "Bob", "Train B", "15B", "2024-08-02"),
320         Booking(3, "Charlie", "Train A", "10C", "2024-08-01"),
321     ]
322
323     # Sort bookings by travel date
324     sorted_by_date = merge_sort_bookings(bookings, key='travel_date')
325     print("Bookings sorted by travel date:")
326     for booking in sorted_by_date:
327         print(f"{booking.passenger_name} on {booking.train_number} seat {booking.seat_number} for {booking.travel_date}")
328
329     # Search for a booking by ticket ID
330     target_id = 2
331     found_booking = binary_search_bookings(sorted_by_date, target_id)
332     if found_booking:
333         print(f"\nBooking found: {found_booking.passenger_name} on {found_booking.train_number} seat {found_booking.seat_number} for {found_booking.travel_date}")
334     else:
335         print(f"\nBooking with ID {target_id} not found.")
336

```

OUTPUT:

```

● Bookings sorted by travel date:
Charlie on Train A seat 10C for 2024-08-01
Alice on Train A seat 12A for 2024-08-01
Bob on Train B seat 15B for 2024-08-02

Booking found: Bob on Train B seat 15B for 2024-08-02
○ PS D:\AI>

```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search helps to find ticket details using ticket ID quickly. It reduces the number of comparisons compared to linear search.

Sorting Algorithm: Merge Sort

Merge sort is used to sort booking details based on travel date or seat number. It works efficiently even when the number of bookings increases.

Task 5 – Smart Hostel Room Allocation System**Prompt :**

#Smart Hostel Room Allocation System

A hostel management system stores student room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

CODE:

```

327
328 #Smart Hostel Room Allocation System
329 # A hostel management system stores student room allocation details
330 # including student ID, room number, floor, and allocation date. The
331 # system needs to:
332 # 1. Search allocation details using student ID.
333 # 2. Sort records based on room number or allocation date.
334 # suggest optimized algorithms.
335 # • Justify the selections.
336 # • Implement the solution in Python.
337 '''For searching allocation details using student ID, a suitable algorithm would be Binary Search, provided that the allocation records are sorted by student ID. Binary Search is efficient with a time complexity of  $O(\log n)$ . For sorting records based on room number or allocation date, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of  $O(n \log n)$  and is stable, meaning it preserves the relative order of equal elements. Here is the implementation of both algorithms in Python:'''
338
339 class Allocation:
340     def __init__(self, student_id, room_number, floor, allocation_date):
341         self.student_id = student_id
342         self.room_number = room_number
343         self.floor = floor
344         self.allocation_date = allocation_date
345
346     def binary_search_allocations(allocations, target_id):
347         """
348         Searches for an allocation by student ID using Binary Search.
349
350         Time Complexity:  $O(\log n)$  - where n is the number of allocations.
351         Space Complexity:  $O(1)$  - as it uses constant extra space.
352
353         Parameters:
354         allocations (list): A list of Allocation objects sorted by student_id.
355         target_id: The student ID to search for.
356
357         Returns:
358         Allocation: The allocation object if found, otherwise None.
359         """
360         left, right = 0, len(allocations) - 1
361
362         while left <= right:
363             mid = left + (right - left) // 2
364
365             if allocations[mid].student_id == target_id:
366                 return allocations[mid]
367             elif allocations[mid].student_id < target_id:
368                 left = mid + 1
369             else:
370                 right = mid - 1
371
372         return None
373
374     def merge_sort_allocations(allocations, key):
375         """Sorts a list of Allocation objects based on a specified key using Merge Sort.

```

```

376
377         def merge_sort_allocations(allocations, key):
378             """Sorts a list of Allocation objects based on a specified key using Merge Sort.
379             Time Complexity:  $O(n \log n)$  - where n is the number of allocations.
380             Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
381             Parameters:
382             allocations (list): A list of Allocation objects to be sorted.
383             key (str): The attribute to sort by ('room_number' or 'allocation_date').
384             Returns:
385             list: A new sorted list of Allocation objects based on the specified key.
386             """
387             if len(allocations) <= 1:
388                 return allocations
389
390             mid = len(allocations) // 2
391             left_half = merge_sort_allocations(allocations[:mid], key)
392             right_half = merge_sort_allocations(allocations[mid:], key)
393
394             return merge_allocations(left_half, right_half, key)
395
396         def merge_allocations(left, right, key):
397             """Merges two sorted lists of Allocation objects based on a specified key.
398             Parameters:
399             left (list): The first sorted list of Allocation objects.
400             right (list): The second sorted list of Allocation objects.
401             key (str): The attribute to sort by ('room_number' or 'allocation_date').
402             Returns:
403             list: A merged sorted list of Allocation objects based on the specified key.
404             """
405             merged = []
406             left_index = right_index = 0
407
408             while left_index < len(left) and right_index < len(right):
409                 if getattr(left[left_index], key) < getattr(right[right_index], key):
410                     merged.append(left[left_index])
411                     left_index += 1
412                 else:
413                     merged.append(right[right_index])
414                     right_index += 1
415
416             while left_index < len(left):
417                 merged.append(left[left_index])
418                 left_index += 1
419
420             while right_index < len(right):
421                 merged.append(right[right_index])
422                 right_index += 1
423
424             return merged

```



```

421 while left_index < len(left):
422     merged.append(left[left_index])
423     left_index += 1
424
425 while right_index < len(right):
426     merged.append(right[right_index])
427     right_index += 1
428
429 return merged
430
431 # Example usage:
432 if __name__ == "__main__":
433     allocations = [
434         Allocation(1, "101", "1st Floor", "2024-09-01"),
435         Allocation(2, "102", "1st Floor", "2024-09-02"),
436         Allocation(3, "201", "2nd Floor", "2024-09-01"),
437     ]
438     # Sort allocations by room number
439     sorted_by_room = merge_sort_allocations(allocations, key='room_number')
440     print("Allocations sorted by room number:")
441     for alloc in sorted_by_room:
442         print(f"Student ID: {alloc.student_id}, Room: {alloc.room_number}, Floor: {alloc.floor}, Date: {alloc.allocation_date}")
443
444     # Search for an allocation by student ID
445     target_id = 2
446     found_allocation = binary_search_allocations(sorted_by_room, target_id)
447     if found_allocation:
448         print(f"Allocation found: Student ID: {found_allocation.student_id}, Room: {found_allocation.room_number}, Floor: {found_allocation.floor}, Date: {found_allocation.allocation_date}")
449     else:
450         print(f"Allocation with Student ID {target_id} not found.")
451
452

```

OUTPUT:

```

• Allocations sorted by room number:
  Student ID: 1, Room: 101, Floor: 1st Floor, Date: 2024-09-01
  Student ID: 2, Room: 102, Floor: 1st Floor, Date: 2024-09-02
  Student ID: 3, Room: 201, Floor: 2nd Floor, Date: 2024-09-01

  Allocation found: Student ID: 2, Room: 102, Floor: 1st Floor, Date: 2024-09-02
○ PS D:\AI>

```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to search student room allocation using student ID. Since student IDs are unique, the search becomes faster.

Sorting Algorithm: Merge Sort

Merge sort is used to sort records by room number or allocation date. It provides efficient sorting for large data.

Task 6 – Online Movie Streaming Platform

Prompt :

#Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Recommend searching and sorting algorithms .

• Justify the chosen algorithms.

• Implement Python functions.

CODE:

```
454 #Online Movie Streaming Platform
455 # A streaming service maintains movie records with movie ID, title,
456 # genre, rating, and release year. The platform needs to:
457 # 1. Search movies by movie ID.
458 # 2. Sort movies based on rating or release year.
459 # Recommend searching and sorting algorithms.
460 # • Justify the chosen algorithms.
461 # • Implement Python functions.
462 '''For searching movies by movie ID, a suitable algorithm would be Binary Search, provided that the movie records are sorted by movie ID. Binary Search is efficient with a time complexity of  $O(\log n)$ .
463 For sorting movies based on rating or release year, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of  $O(n \log n)$  and is stable, meaning it preserves the relative order of elements.
464 Here is the implementation of both algorithms in Python:'''
465 class Movie:
466     def __init__(self, movie_id, title, genre, rating, release_year):
467         self.movie_id = movie_id
468         self.title = title
469         self.genre = genre
470         self.rating = rating
471         self.release_year = release_year
472 def binary_search_movies(movies, target_id):
473     """
474     Searches for a movie by movie ID using Binary Search.
475
476     Time Complexity:  $O(\log n)$  - where  $n$  is the number of movies.
477     Space Complexity:  $O(1)$  - as it uses constant extra space.
478
479     Parameters:
480     movies (list): A list of Movie objects sorted by movie_id.
481     target_id: The movie ID to search for.
482
483     Returns:
484     Movie: The movie object if found, otherwise None.
485     """
486     left, right = 0, len(movies) - 1
487
488     while left <= right:
489         mid = left + (right - left) // 2
490
491         if movies[mid].movie_id == target_id:
492             return movies[mid]
493         elif movies[mid].movie_id < target_id:
494             left = mid + 1
495         else:
496             right = mid - 1
497
498     return None
499 def merge_sort_movies(movies, key):
500     """Sorts a list of Movie objects based on a specified key using Merge Sort.
```

```
499 def merge_sort_movies(movies, key):
500     """Sorts a list of Movie objects based on a specified key using Merge Sort.
501     Time Complexity:  $O(n \log n)$  - where  $n$  is the number of movies.
502     Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
503     Parameters:
504     movies (list): A list of Movie objects to be sorted.
505     key (str): The attribute to sort by ('rating' or 'release_year').
506     Returns:
507     list: A new sorted list of Movie objects based on the specified key.
508     """
509     if len(movies) <= 1:
510         return movies
511
512     mid = len(movies) // 2
513     left_half = merge_sort_movies(movies[:mid], key)
514     right_half = merge_sort_movies(movies[mid:], key)
515
516     return merge_movies(left_half, right_half, key)
517 def merge_movies(left, right, key):
518     """Merges two sorted lists of Movie objects based on a specified key.
519     Parameters:
520     left (list): The first sorted list of Movie objects.
521     right (list): The second sorted list of Movie objects.
522     key (str): The attribute to sort by ('rating' or 'release_year').
523     Returns:
524     list: A merged sorted list of Movie objects based on the specified key.
525     """
526     merged = []
527     left_index = right_index = 0
528
529     while left_index < len(left) and right_index < len(right):
530         if getattr(left[left_index], key) < getattr(right[right_index], key):
531             merged.append(left[left_index])
532             left_index += 1
533         else:
534             merged.append(right[right_index])
535             right_index += 1
536
537     while left_index < len(left):
538         merged.append(left[left_index])
539         left_index += 1
540
541     while right_index < len(right):
542         merged.append(right[right_index])
543         right_index += 1
544
545     return merged
```

```

536
537     while left_index < len(left):
538         merged.append(left[left_index])
539         left_index += 1
540
541     while right_index < len(right):
542         merged.append(right[right_index])
543         right_index += 1
544
545     return merged
546
547 # Example usage:
548 if __name__ == "__main__":
549     movies = [
550         Movie(1, "Movie A", "Action", 8.5, 2020),
551         Movie(2, "Movie B", "Comedy", 7.2, 2019),
552         Movie(3, "Movie C", "Drama", 9.0, 2021),
553     ]
554     # Sort movies by rating
555     sorted_by_rating = merge_sort_movies(movies, key='rating')
556     print("Movies sorted by rating:")
557     for movie in sorted_by_rating:
558         print(f"{movie.title} ({movie.genre}) - Rating: {movie.rating}, Release Year: {movie.release_year}")
559
560     # Search for a movie by ID
561     target_id = 2
562     found_movie = binary_search_movies(sorted_by_rating, target_id)
563     if found_movie:
564         print(f"\nMovie found: {found_movie.title} ({found_movie.genre}) - Rating: {found_movie.rating}, Release Year: {found_movie.release_year}")
565     else:
566         print(f"\nMovie with ID {target_id} not found.")
567
568

```

OUTPUT:

```

Movies sorted by rating:
Movie B (Comedy) - Rating: 7.2, Release Year: 2019
Movie A (Action) - Rating: 8.5, Release Year: 2020
Movie C (Drama) - Rating: 9.0, Release Year: 2021

Movie with ID 2 not found.
○ PS D:\AI>

```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search helps to quickly find movies using movie ID in the system.

Sorting Algorithm: Merge Sort

Merge sort is used to sort movies based on rating or release year because it works efficiently for large movie databases.

Task 7 – Smart Agriculture Crop Monitoring System

Prompt :

#Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop

name, soil moisture level, temperature, and yield estimate. Farmers

need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

select algorithms.

• Justify algorithm suitability.

• Implement searching and sorting in Python.

CODE:

```
571 #Smart Agriculture Crop Monitoring System
572 # An agriculture monitoring system stores crop data with crop ID, crop
573 # name, soil moisture level, temperature, and yield estimate. Farmers
574 # need to:
575 # 1. Search crop details using crop ID.
576 # 2. Sort crops based on moisture level or yield estimate.
577 # select algorithms.
578 # • Justify algorithm suitability.
579 # • Implement searching and sorting in Python.
580 '''For searching crop details using crop ID, a suitable algorithm would be Binary Search, provided that the crop records are sorted by crop ID. Binary Search is efficient with a time complexity of  $O(\log n)$  and is stable, meaning it preserves the relative order of elements.
581 For sorting crops based on moisture level or yield estimate, the Merge Sort algorithm is a suitable choice. Merge Sort has a time complexity of  $O(n \log n)$  and is stable, meaning it preserves the relative order of elements.
582 Here is the implementation of both algorithms in Python:'''
583 class Crop:
584     def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
585         self.crop_id = crop_id
586         self.crop_name = crop_name
587         self.soil_moisture_level = soil_moisture_level
588         self.temperature = temperature
589         self.yield_estimate = yield_estimate
590 def binary_search_crops(crops, target_id):
591     """
592     Searches for a crop by crop ID using Binary Search.
593
594     Time Complexity:  $O(\log n)$  - where n is the number of crops.
595     Space Complexity:  $O(1)$  - as it uses constant extra space.
596
597     Parameters:
598     crops (list): A list of Crop objects sorted by crop_id.
599     target_id: The crop ID to search for.
600
601     Returns:
602     Crop: The crop object if found, otherwise None.
603     """
604     left, right = 0, len(crops) - 1
605     while left <= right:
606         mid = left + (right - left) // 2
607         if crops[mid].crop_id == target_id:
608             return crops[mid]
609         elif crops[mid].crop_id < target_id:
610             left = mid + 1
611         else:
612             right = mid - 1
613     return None
614 def merge_sort_crops(crops, key):
```

```
618     """Sorts a list of Crop objects based on a specified key using Merge Sort.
619     Time Complexity:  $O(n \log n)$  - where n is the number of crops.
620     Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
621     Parameters:
622     crops (list): A list of Crop objects to be sorted.
623     key (str): The attribute to sort by ('soil_moisture_level' or 'yield_estimate').
624     Returns:
625     list: A new sorted list of Crop objects based on the specified key.
626     """
627     if len(crops) <= 1:
628         return crops
629     mid = len(crops) // 2
630     left_half = merge_sort_crops(crops[:mid], key)
631     right_half = merge_sort_crops(crops[mid:], key)
632     return merge_crops(left_half, right_half, key)
633 def merge_crops(left, right, key):
634     """Merges two sorted lists of Crop objects based on a specified key.
635     Parameters:
636     left (list): The first sorted list of Crop objects.
637     right (list): The second sorted list of Crop objects.
638     key (str): The attribute to sort by ('soil_moisture_level' or 'yield_estimate').
639     Returns:
640     list: A merged sorted list of Crop objects based on the specified key.
641     """
642     merged = []
643     left_index = 0
644     right_index = 0
645     while left_index < len(left) and right_index < len(right):
646         if getattr(left[left_index], key) < getattr(right[right_index], key):
647             merged.append(left[left_index])
648             left_index += 1
649         else:
650             merged.append(right[right_index])
651             right_index += 1
652     while left_index < len(left):
653         merged.append(left[left_index])
654         left_index += 1
655     while right_index < len(right):
656         merged.append(right[right_index])
657         right_index += 1
658     return merged
```

```

655     while left_index < len(left):
656         merged.append(left[left_index])
657         left_index += 1
658
659     while right_index < len(right):
660         merged.append(right[right_index])
661         right_index += 1
662
663     return merged
664
665 # Example usage:
666 if __name__ == "__main__":
667     crops = [
668         Crop(1, "Wheat", 30, 25, 1000),
669         Crop(2, "Corn", 40, 30, 1500),
670         Crop(3, "Rice", 35, 28, 1200),
671     ]
672     # Sort crops by soil moisture level
673     sorted_by_moisture = merge_sort_crops(crops, key='soil_moisture_level')
674     print("Crops sorted by soil moisture level:")
675     for crop in sorted_by_moisture:
676         print(f"{crop.crop_name} - Moisture Level: {crop.soil_moisture_level}, Yield Estimate: {crop.yield_estimate}")
677
678     # Search for a crop by ID
679     target_id = 2
680     found_crop = binary_search_crops(sorted_by_moisture, target_id)
681     if found_crop:
682         print(f"\nCrop found: {found_crop.crop_name} - Moisture Level: {found_crop.soil_moisture_level}, Yield Estimate: {found_crop.yield_estimate}")
683     else:
684         print(f"\nCrop with ID {target_id} not found.")
685
686
687
688
689 # Airport Flight Management System

```

OUTPUT:

```

Crops sorted by soil moisture level:
Wheat - Moisture Level: 30, Yield Estimate: 1000
Rice - Moisture Level: 35, Yield Estimate: 1200
Corn - Moisture Level: 40, Yield Estimate: 1500

Crop with ID 2 not found.
PS D:\AI>

```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to search crop details using crop ID quickly.

Sorting Algorithm: Merge Sort

Merge sort helps to sort crop data based on moisture level or yield estimate in an efficient way.

Task 8 – Airport Flight Management System

Prompt :

#Airport Flight Management System

An airport system stores flight information including flight ID, #
airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

#select other than merge sort

• Justify the algorithm choices.

• Implement the solution in Python.

CODE:

```
689 #Airport Flight Management System
690 # An airport system stores flight information including flight ID,
691 # airline name, departure time, arrival time, and status. The system
692 # must:
693 # 1. Search flight details using flight ID.
694 # 2. Sort flights based on departure time or arrival time.
695 #select other than merge sort
696 # • Justify the algorithm choices.
697 # • Implement the solution in Python.
698 '''For searching flight details using flight ID, a suitable algorithm would be Binary Search, provided that
699 the flight records are sorted by flight ID. Binary Search is efficient with a time complexity of  $O(\log n)$  and is ideal for large datasets, making it a good choice for searching through flight records.
700 For sorting flights based on departure time or arrival time, the Quick Sort algorithm is a suitable choice. Quick Sort has an average time complexity of  $O(n \log n)$  and is an in-place sorting algorithm.
701 Here is the implementation of both algorithms in Python:'''
702 class Flight:
703     def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
704         self.flight_id = flight_id
705         self.airline_name = airline_name
706         self.departure_time = departure_time
707         self.arrival_time = arrival_time
708         self.status = status
709     def binary_search_flights(flights, target_id):
710         """
711         Searches for a flight by flight ID using Binary Search.
712
713         Time Complexity:  $O(\log n)$  - where  $n$  is the number of flights.
714         Space Complexity:  $O(1)$  - as it uses constant extra space.
715
716         Parameters:
717         flights (list): A list of Flight objects sorted by flight_id.
718         target_id: The flight ID to search for.
719
720         Returns:
721         Flight: The Flight object if found, otherwise None.
722         """
723         left, right = 0, len(flights) - 1
724         while left <= right:
725             mid = left + (right - left) // 2
726             if flights[mid].flight_id == target_id:
727                 return flights[mid]
728             elif flights[mid].flight_id < target_id:
729                 left = mid + 1
730             else:
731                 right = mid - 1
732         return None
```

```
736 def quick_sort_flights(flights, key):
737     """Sorts a list of Flight objects based on a specified key using Quick Sort.
738     Time Complexity:  $O(n \log n)$  on average,  $O(n^2)$  in the worst case.
739     Space Complexity:  $O(\log n)$  on average,  $O(n)$  in the worst case due to recursion.
740     Parameters:
741     flights (list): A list of Flight objects to be sorted.
742     key (str): The attribute to sort by ('departure_time' or 'arrival_time').
743     Returns:
744     list: A new sorted list of Flight objects based on the specified key.
745     """
746     if len(flights) <= 1:
747         return flights
748     pivot = flights[len(flights) // 2]
749     left = [x for x in flights if getattr(x, key) < getattr(pivot, key)]
750     middle = [x for x in flights if getattr(x, key) == getattr(pivot, key)]
751     right = [x for x in flights if getattr(x, key) > getattr(pivot, key)]
752     return quick_sort_flights(left, key) + middle + quick_sort_flights(right, key)
753
754 # Example usage:
755 if __name__ == "__main__":
756     flights = [
757         Flight(1, "Airline A", "2024-10-01 08:00", "2024-10-01 10:00", "On Time"),
758         Flight(2, "Airline B", "2024-10-01 09:00", "2024-10-01 11:00", "Delayed"),
759         Flight(3, "Airline C", "2024-10-01 07:00", "2024-10-01 09:00", "On Time"),
760     ]
761     # Sort flights by departure time
762     sorted_by_departure = quick_sort_flights(flights, key='departure_time')
763     print("Flights sorted by departure time:")
764     for flight in sorted_by_departure:
765         print(f"{flight.airline_name} - Departure: {flight.departure_time}, Arrival: {flight.arrival_time}, Status: {flight.status}")
766
767     # Search for a flight by ID
768     target_id = 2
769     found_flight = binary_search_flights(sorted_by_departure, target_id)
770     if found_flight:
771         print(f"\nFlight found: {found_flight.airline_name} - Departure: {found_flight.departure_time}, Arrival: {found_flight.arrival_time}, Status: {found_flight.status}")
772     else:
773         print(f"\nFlight with ID {target_id} not found.")
774
775
776
```

OUTPUT:

```
● Flights sorted by departure time:
Airline C - Departure: 2024-10-01 07:00, Arrival: 2024-10-01 09:00, Status: On Time
Airline A - Departure: 2024-10-01 08:00, Arrival: 2024-10-01 10:00, Status: On Time
Airline B - Departure: 2024-10-01 09:00, Arrival: 2024-10-01 11:00, Status: Delayed

Flight found: Airline B - Departure: 2024-10-01 09:00, Arrival: 2024-10-01 11:00, Status: Delayed
○ PS D:\AI> █
```

JUSTIFICATION:

Searching Algorithm: Binary Search

Binary search is used to find flight details using flight ID quickly.

Sorting Algorithm: Merge Sort

Merge sort is used to sort flights based on departure time or arrival time efficiently.